Custom agent boundaries

Fail the weak profile, tighten it, rerun the same test

Session M3-04 | 3 hours

Agenda

Segment	Time
What a profile controls, and when it beats a general agent	6 min
`.agent.md` frontmatter and tool aliases	10 min
How an agent is selected; tools, instructions, model	7 min
Weak profile and boundary violation	9 min
Least-privilege tools and paths	9 min
Stop and human handoff	6 min
Rerun the contract	5 min
Agent vs skill vs MCP	6 min
Lab handoff	2 min

A profile is a role contract

```
Profile
  role + tools + procedure + stop rules

Environment
  actual permissions + repository access

Human
  accept + revise + reject
```

The profile guides behavior. It doesn't grant authority.

Custom agent vs general agent

A **custom agent** is a saved role: instructions, a tool list, and optionally a model, in one Markdown file the team can review.

Use the general agent	Use a custom agent
One-off or exploratory work	The same role, again and again
Rules fit in repository instructions	The role needs **fewer tools** than the default
You steer every step	It needs a **stop rule** or a fixed procedure
One person, one session	A team should get **the same behavior**

If instructions are enough, don't make an agent.

Repository profile structure

```
.github/
└── agents/
    └── test-writer.agent.md
```

YAML frontmatter plus a Markdown body. The filename identifies the profile.

Frontmatter used in the lab

```
---
name: Test-only writer
description: Adds focused JavaScript tests in tests/ and stops when production code must change.
target: github-copilot
tools:
  - read
  - search
  - edit
  - execute
disable-model-invocation: true
user-invocable: true
---
```

Frontmatter, field by field

Field	What it does
`description`	**Required.** Says what the agent does. Copilot reads it to pick the agent
`name`	Display name. Optional
`tools`	Which tools the agent can use. **Omit it and you get all tools**
`model`	Model for this agent. Unset inherits the default
`target`	`vscode` or `github-copilot`. Unset means both
`disable-model-invocation`	`true`: Copilot never picks this agent on its own. A person must select it
`user-invocable`	`false`: people can't select it. Default `true`
`mcp-servers`	Extra MCP servers for this agent

`infer` is retired. Use the two fields above. **Check current docs**: fields differ by client, and `metadata` and `mcp-servers` don't apply in IDE agents.

Tool aliases

`tools` takes aliases. They are case-insensitive.

Alias	Allows
`read`	Read file contents
`search`	Find files or text
`edit`	Edit files
`execute`	Run shell commands
`agent`	Call another custom agent
`web`	Fetch URLs and search (not in the cloud agent today)

- `tools: ["*"]` or no `tools`: everything, including MCP tools.
- `tools: []`: nothing.
- MCP tools: `server-name/tool` or `server-name/*`.
- Unknown names are ignored. A typo fails quietly.

How an agent is selected

Way	How
Pick it	`/agent` in the CLI, or `copilot --agent=NAME`
Name it	"Use the test-writer agent to..."
Copilot picks it	The model may delegate to a matching agent, based on `description`

Set `disable-model-invocation: true` when a wrong auto-pick is costly. Then only a person starts it.

Where agents live: `.github/agents/` (repository), `~/.copilot/agents` (user), or the organization's `.github` repository. On a name clash, user beats repository, and repository beats organization.

What changes the agent's behavior

Setting	Effect	Failure when wrong
Tools	What it can do at all	Too many: it can edit anything
Instructions (body)	How it works, where it stops	Vague: it guesses
Model	Reasoning quality, speed, cost	Too weak for the task, or too costly

Tools set the ceiling. Instructions steer inside it. **Neither replaces review.** The environment still controls real permissions.

The weak profile looks convenient

```
tools: ["*"]
```

```
Add tests for the requested behavior.
Read the code, make any changes needed, run the tests, and report the result.
```

It never says which files may change or when to stop.

Test a source-code conflict

```
Add tests for updateOrderStatus.
Change tests only.

A new criterion allows reversing shipped to processing.
The current implementation rejects that transition.
```

The weak wording permits a source edit. One lucky run in scope doesn't fix the contract.

Make the failure repeatable

```
npm run check:weak
```

Expected failures:

- every tool is enabled;
- no test-only path is named;
- no production-code stop exists;
- no test command or human review decision is named.

Tool allowlists are necessary

Need	Alias
Read source and tests	`read`
Find conventions	`search`
Edit test files	`edit`
Run the known test command	`execute`

But `edit` still needs a path rule in the body.

The body sets the path boundary

Allowed:

```
Read src/
Search the repository
Edit tests/
Run npm test
```

Forbidden:

```
Edit src/
Change manifests or workflows
Add dependencies
Push or merge
```

Stop instead of crossing the boundary

```
No files changed.
Blocked: the requested assertion requires a production-code change under src/.
Decision needed: approve a separate source change or revise the criteria.
```

The stop report names the conflict and the decision a human must make.

Rerun the same contract

```
npm run check:tight
npm test
```

Expected:

```
PASS .../test-writer.agent.md
tests 3
pass 3
fail 0
```

The tightened profile passes the contract that failed the weak one.

Agent, skill, or MCP?

	Skill	Custom agent	MCP server
Gives Copilot	Know-how	A role	Reach
Example	`release-notes`: your changelog format	`test-writer`: edits `tests/` only, runs `npm test`	GitHub or Microsoft Learn server: live data
Loads	On demand	When picked or delegated	When configured
Risk	Wrong trigger	Too many tools	Credentials, data

Pick by the need:

- "Copilot should **know how** we do this" → skill.
- "Copilot should **act as** this role, with these limits" → custom agent.
- "Copilot should **reach** this system" → MCP server.
- "This always applies" → instructions.

They combine. A custom agent can use a skill and list the MCP tools it may call. Don't add MCP to a role that only needs repository files. More: M3-03, M3-05.

Optional stretch: check code intelligence

```
/lsp show
/lsp test typescript
```

Ask for the definition and references of `getOrdersByUser`. Inspect the LSP call or logs, then check `getUserOrderStats` in the source.

An export is not a caller. A startup test is not a symbol lookup. Don't edit code in this step.

Lab handoff

1. Inspect the `.agent.md` structure.
2. Run the weak contract.
3. Record the source edit the weak profile permits.
4. Tighten tools, paths, validation, and stop behavior.
5. Rerun the contract.
6. Produce a no-change stop report.

Submit both the failed weak profile and the passing tightened profile, plus the live stop. A manual trace doesn't complete it. The LSP lookup is an optional stretch.